



Departamento de Informática e Matemática Aplicada – DIMAp

Comunicação TCP

Prof. Nélio Cacho

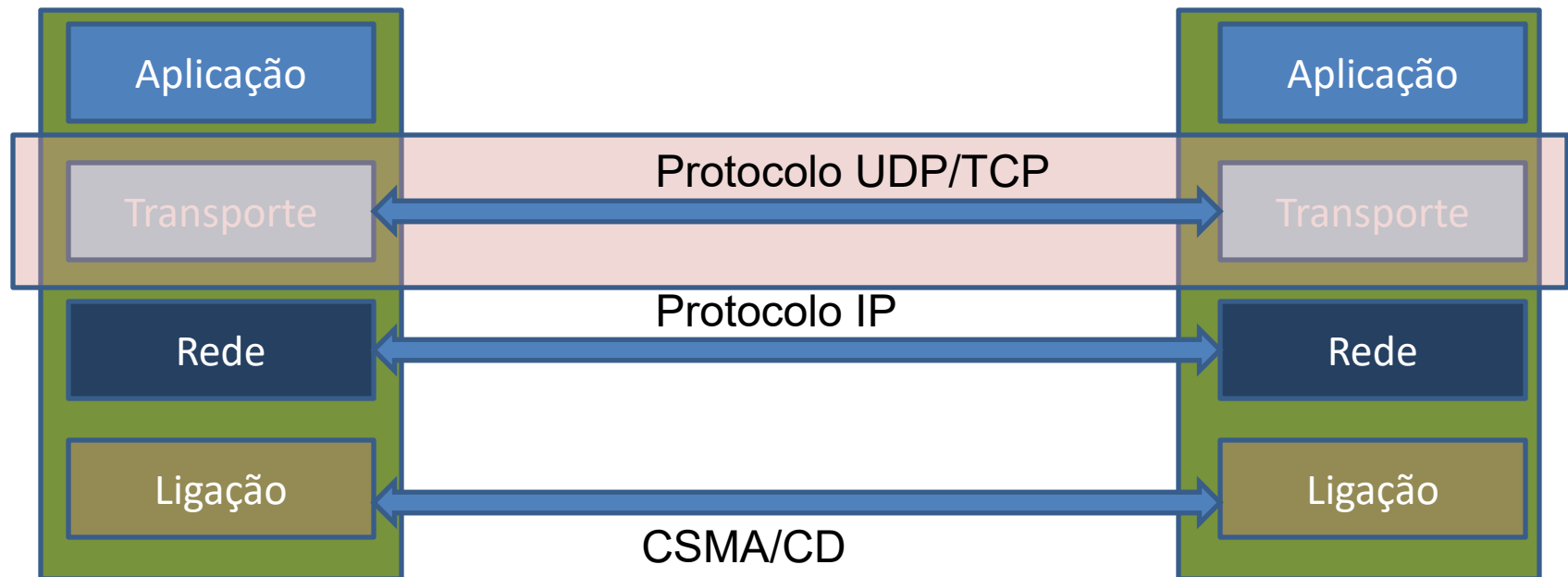
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Como realizar a comunicação ?



Vantagens do TCP

- Controle automático de erro
- Confiabilidade – Dados são entregues na ordem em que são enviados
- Facilidade de Uso: Não existe a necessidade de enviar pacotes de dados separadamente.

Interação TCP Servidor

Servidor começa por criar um TCP Server Socket...

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
ServerSocket servSock = new ServerSocket(servPort);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
for (;;) {  
    Socket clntSock = servSock.accept();
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

Servidor fica bloqueado esperando conexões

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente

Um cliente então resolve estabelecer uma comunicação com o servidor...

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente

```
Socket socket = new Socket(server, servPort);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente

```
OutputStream out = socket.getOutputStream();  
out.write(byteBuffer);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
Socket clntSock = servSock.accept();
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
InputStream in = clntSock.getInputStream();  
recvMsgSize = in.read(byteBuffer);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente/Servidor

`close(sock);`

`close(clntSocket)`

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Enviando Instancias de Classes

```
public class MsgTCP implements Serializable{  
    private int proc;  
    private int mem;  
    private int nclients;  
    private String hostname;  
}
```

Enviando Instancias de Classes

```
Socket conexao = new Socket ("localhost", SERVICE_PORT);
```

```
ObjectOutputStream output = new  
    ObjectOutputStream(conexao.getOutputStream());
```

```
MsgTCP msg = new MsgTCP();
```

```
msg.setMem(20);
```

```
output.writeObject(msg);
```

```
output.flush();
```


Exemplo Cliente

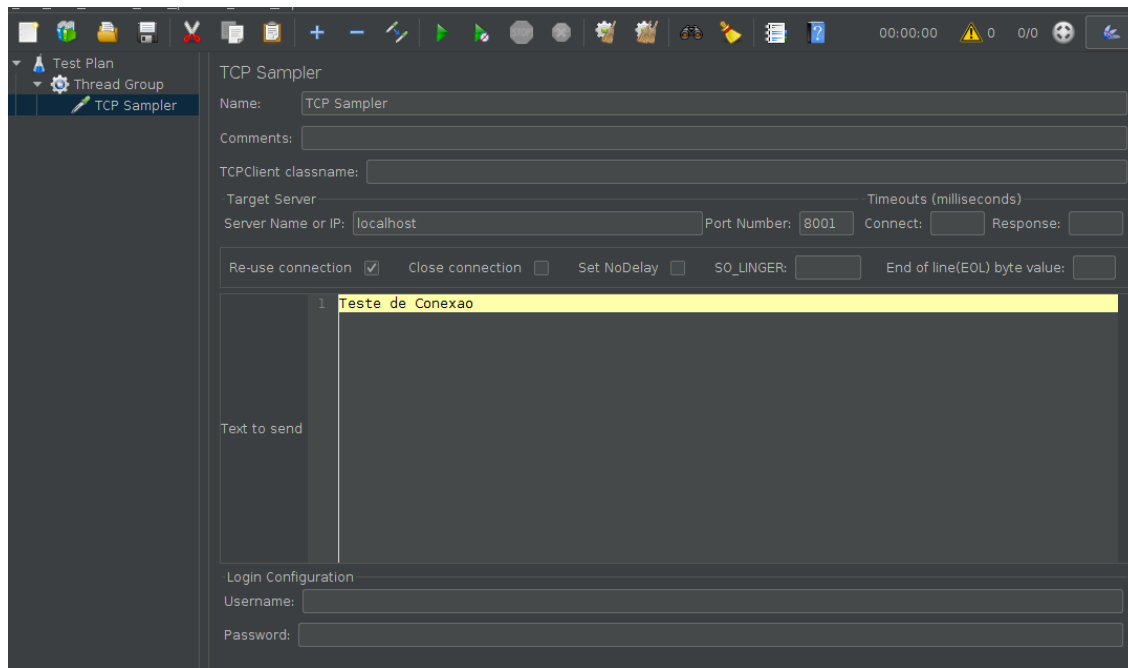
```
public class TCPClient {  
  
    public static void main(String args[]){  
        Socket conexao = null;  
        try {  
            conexao = new Socket ("localhost", 8080);  
            ObjectOutputStream output = new  
                ObjectOutputStream(conexao.getOutputStream());  
            MsgTCP msg = new MsgTCP();  
            msg.setMem(20);  
            output.writeObject(msg);  
            output.flush();  
        } catch (UnknownHostException e) {  
            e.printStackTrace();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }finally {  
            try {  
                conexao.close();  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

Exemplo Servidor

```
public class TCPServer {
    public static void main(String args[]){
        ServerSocket server = null;
        try {
            server = new ServerSocket (8080);
        } catch (IOException e2) {
            e2.printStackTrace();
        }
        while(true)
        {
            try{
                Socket nextClient = server.accept();
                ObjectInputStream input = new ObjectInputStream(nextClient.getInputStream());
                MsgTCP msg = (MsgTCP)input.readObject();
                System.out.println("Proc:"+msg.getProc()+" Mem:"+msg.getMem()+" Num. Cli:"+msg.getNclients()+" host:"+msg.getHostname());
            } catch (ClassNotFoundException e) {
                e.printStackTrace();
                try {
                    server.close();
                } catch (IOException e1) {
                    e1.printStackTrace();
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Testar Serviço TCP com JMeter

- Para criar um Sampler TCP, clicar com botão direito no ThreadGroup, "add"-> "Sampler"-> "TCP Sampler".



Criar Sampler TCP

Test Plan
Thread Group
TCP Sampler

TCP Sampler

Name: TCP Sampler

Comments:

TCPClient classname:

Target Server

Server Name or IP: localhost Port Number: 8001

Re-use connection ☒ Close connection ☐ Set NoDelay ☐ SO_LINGER: End of line(EOL) byte value:

1
2

Text to send

Login Configuration

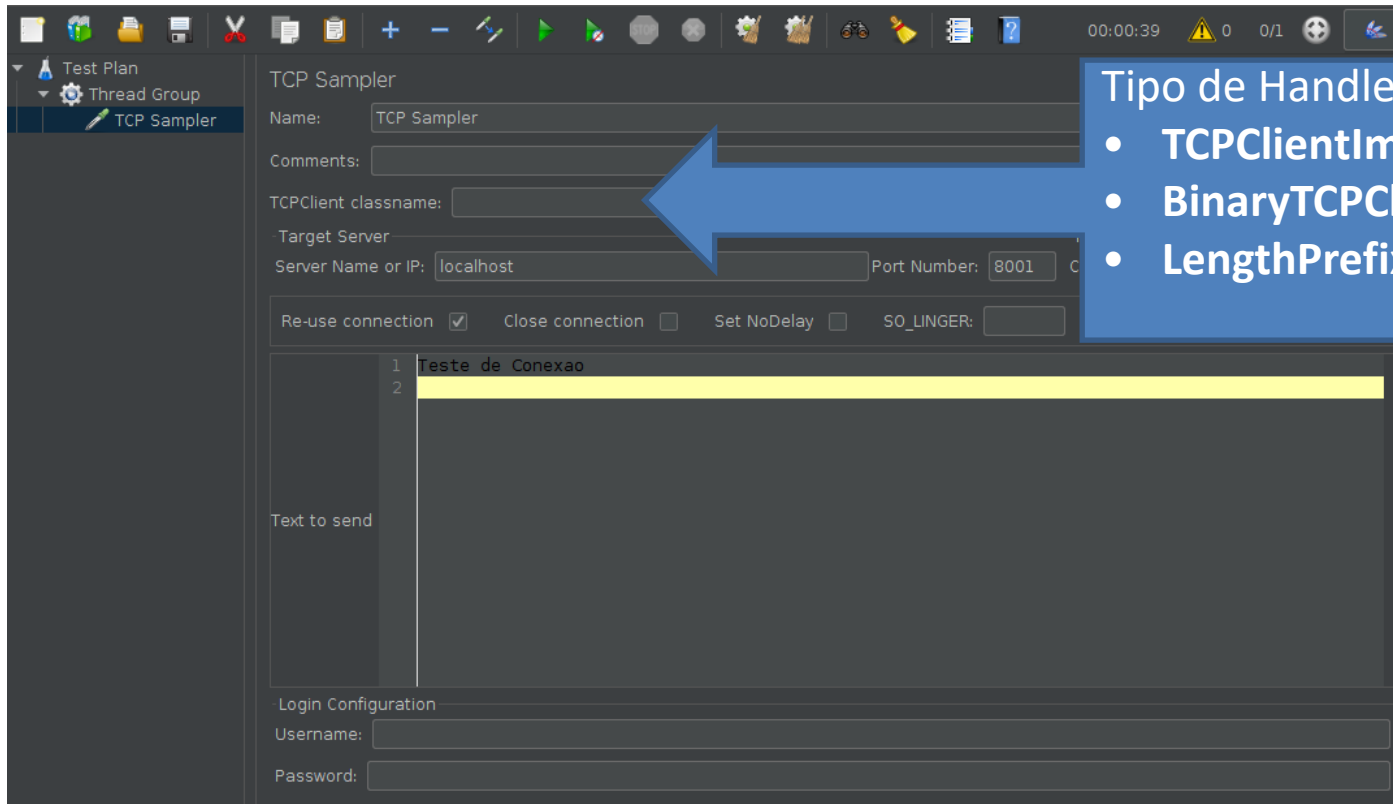
Username:

Password:

Endereço e Porta que o servidor TCP foi iniciado

Colocar dado a ser enviado

Criar Sampler TCP



Tipo de Handler:

- TCPClientImpl (padrão)
- BinaryTCPClientImpl
- LengthPrefixedBinaryTCPClientImpl

Criar Sampler TCP

- Um novo Handler "must extend":
 - **org.apache.jmeter.protocol.tcp.sampler.TCPClient**

TCPClientImpl

This implementation is fairly basic. When reading the response, it reads until the end of line byte, if this is defined by setting the property **tcp.eolByte**, otherwise until the end of the input stream. You can control charset encoding by setting **tcp.charset**, which will default to Platform default encoding.

BinaryTCPClientImpl

This implementation converts the GUI input, which must be a hex-encoded string, into binary, and performs the reverse when reading the response. When reading the response, it reads until the end of message byte, if this is defined by setting the property **tcp.BinaryTCPClient.eomByte**, otherwise until the end of the input stream.

LengthPrefixedBinaryTCPClientImpl

This implementation extends **BinaryTCPClientImpl** by prefixing the binary message data with a binary length byte. The length prefix defaults to 2 bytes. This can be changed by setting the property **tcp.binarylength.prefix.length**.